Cross-Bank Federated Transaction Fraud Detection

Zohar Sahar (315144840) | Yonatan Harel (208742593)

Machine Learning Course – Semester Project Report

1. Introduction

The report addresses the critical challenge of real-time credit card fraud detection, where financial institutions must identify rare fraudulent events within massive volumes of legitimate data. While gradient-boosted decision trees (specifically XGBoost) are the industry standard, banks are limited by a "siloed" view of data due to strict privacy regulations like GDPR and CCPA, which prohibit sharing raw customer information.

The proposed solution is Federated Learning (FL). This framework allows institutions to collaborate by sharing only model updates rather than raw data. The project simulates a multi-bank environment using NVIDIA FLARE (NVFlare) to determine if federated collaboration provides a performance "uplift" over local models, particularly for institutions with limited fraud data.

2. Project Objectives

- Non-IID Simulation: Partition the IEEE-CIS dataset into three distinct "banks" with heterogeneous fraud distributions.
- Baseline Establishment: Develop optimized, standalone XGBoost models for each bank to serve as performance benchmarks.
- Federated Implementation: Deploy a Cyclic Federated XGBoost pipeline where a global model is iteratively refined across all participating banks.
- Performance Analysis: Quantify the collaborative benefit, focusing on whether data-scarce banks (Bank A) gain significantly from the collective knowledge.
- Reproducibility: Utilize Docker containers to ensure deterministic and repeatable results.

3. Dataset and Partitioning

The study utilizes the IEEE-CIS Fraud Detection dataset, characterized by a significant class imbalance (3.5% fraud rate) across 590,540 transactions.

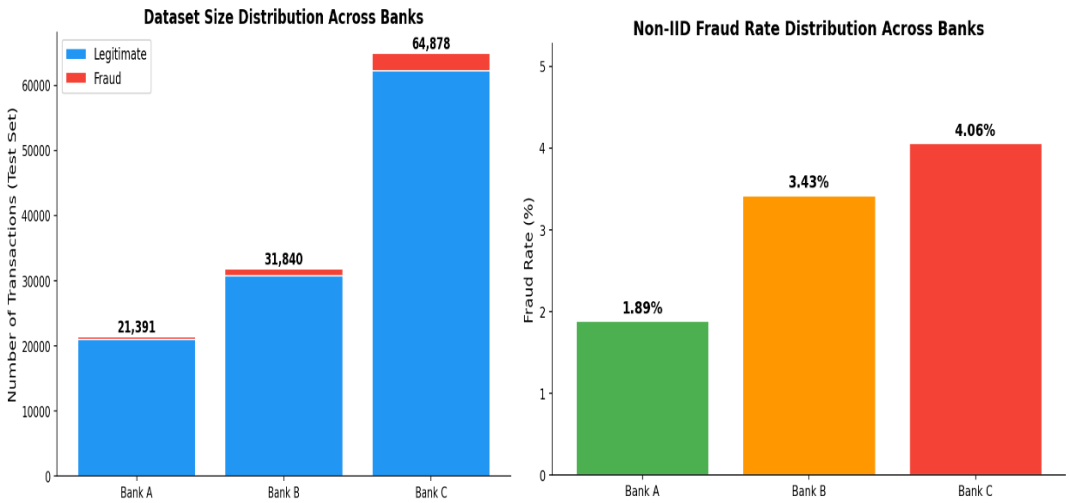
3.1 Partitioning Strategy

To mirror real-world constraints, data was partitioned by card1 (hashed card identifier), ensuring a customer's entire transaction history remains within a single bank. The researchers deliberately introduced non-IID (non-independent and identically distributed) characteristics by skewing the distribution of fraud-bearing cards.

Table 1: Data Distribution and Fraud Profiles per Bank

Bank	Transactions	Fraud Rate	Fraud Profile
Bank A	107,000	1.90%	Low-fraud (data-scarce for fraud)
Bank B	159,000	3.40%	Medium-fraud (balanced)
Bank C	324,000	4.10%	High-fraud (data-rich for fraud)

This setup is designed to test the hypothesis that Bank A will experience the most significant improvement through federation by leveraging the richer fraud patterns available to Bank C.



4. Feature Engineering

4.1 Initial Benchmarks & Motivation

The project began with a raw feature set (Transaction amount, Card/Address info, C-counts, D-deltas, and M-flags). While initial AUC-ROC scores reached 0.93, the F1-scores remained below 0.40, indicating a failure to precisely isolate fraud

from the majority class. This prompted the development of the following specialized features.

4.2 Statistical & Temporal Features

- Z-Score Profile: To detect anomalies relative to a user's typical behavior, the amt_zscore was calculated:

$$amt_zscore = \frac{TransactionAmt - \mu_{card1}}{\sigma_{card1}}$$

This identified "card testing" (unusually small amounts) and "drainage" (unusually large amounts) specific to each card1 group.

- Velocity Windows: To capture "burst" patterns typical of compromised cards, sliding-window features were implemented using a two-pointer algorithm to avoid data leakage.
    - Windows: 30-minute and 2-hour intervals.
    - Metrics: Transaction counts (tx_count) and cumulative sums (tx_sum) preceding the current event.

4.3 Dimensionality Reduction: V-Column Groups

The 339 proprietary Vesta features (V1-V339) were reduced to 22 features. By identifying 12 clusters with identical NaN patterns, each group was summarized by its row-wise mean and standard deviation.

Key Finding: The feature vg11_std emerged as the most significant indicator in SHAP analysis (mean absolute SHAP = 0.780), validating that variance within verification signals is a high-utility fraud predictor.

4.4 Client-Level Aggregation

A composite identifier _uid (card1 + addr1) was used to group transactions. While the ID itself was excluded from training to ensure generalization, the following aggregates were included:

- uid_tx_count: Total transactions per client.
- uid_C1_mean through uid_C14_mean: Averaged behavior counts across the client's history.

4.5 Federated Consistency: Global Encoding

A critical challenge in the federated setting was ensuring that categorical features (e.g., ProductCD, emaildomain) shared the same integer mapping

across all banks. Without this, a model trained at Bank A would misinterpret the feature values at Bank B.

- Solution: A pre-partitioning script generated a category_maps.json shared by all participants, ensuring Identical Label Encoding across the federation.

Summary of Engineered Features

Category	Features	Implementation Method
Anomalies	amt_zscore	Grouped by card1
Velocity	tx_count, tx_sum (30m, 2h)	Two-pointer sliding window
V-Groups	vg1_mean/std to vg11_mean/std	NaN-pattern clustering (22 features)
Client Stats	uid_tx_count, uid_C1-C14_mean	card1 + addr1 aggregation

Total Engineered Features: 42 (Final vector size: ~80 features).

5. Model Architecture

5.1 XGBoost Configuration

XGBoost was selected for its superior performance on tabular data and its robust handling of missing values. A critical component of the configuration was the dynamic handling of class imbalance across the non-IID bank distributions.

Table 2: Optimized Hyperparameters

Parameter	Value	Rationale
n_estimators	400	Increased from initial 100 after early stopping analysis
max_depth	6	Balances expressiveness with generalisation
learning_rate	0.1	Standard rate; lower rates (0.01, 0.05) did not justify the training cost
subsample	0.8	Row subsampling for regularisation

colsample_bytree	0.8	Column subsampling for regularisation
eval_metric	aucpr	Area under the precision-recall curve; appropriate for imbalanced data
tree_method	hist	Histogram-based tree construction for efficiency
scale_pos_weight	Dynamic	Computed per bank as (n_negative / n_positive)

Note on scale_pos_weight: Since Bank A (1.9% fraud) and Bank C (4.1% fraud) have different distributions, the weight is calculated locally at each bank to ensure the model appropriately penalizes misclassified fraudulent transactions within that specific local context.

5.2 Federated Learning Strategy: Cyclic Training

Early experimentation revealed that Federated Averaging (FedAvg) is incompatible with tree-based models, as decision tree structures cannot be numerically averaged like neural network weights.

To resolve this, we implemented a Cyclic Training architecture using NVIDIA FLARE (NVFlare). In this setup, the model is passed serially between banks:

1. Server dispatches the current model to Bank A.
2. Bank A performs a "warm-start," adds 50 new trees based on local data, and returns the update.
3. Server forwards the updated model to Bank B, which repeats the process, followed by Bank C.
4. Final Model: After 15 rounds, the ensemble incorporates collective knowledge from all three banks (totaling up to 2,250 trees).

This was implemented via a custom CyclicXGBController and a FraudLearner that handles model serialization through JSON-formatted boosters.

5.3 Infrastructure and Security

The system is architected as a 4-container Docker Compose environment, ensuring a reproducible and isolated simulation of a production environment.

- Server Container: Manages the CyclicXGBController workflow and coordinates communication.
- Client Containers (Banks A, B, C): Each runs an NVFlare client with local data mounted as a read-only volume.

- Security: Utilizing NVFlare's provisioning, the system employs mutual TLS (mTLS) for encrypted and authenticated communication between all nodes.
- Development: The NVFlare Simulator was used for rapid iterative testing, emulating the full server-client protocol on a single machine.

6. Training and Parameter Tuning

6.1 Initial Baseline Analysis

The first local baselines (100 trees, default 0.5 threshold) yielded high AUC-ROC scores (0.94–0.96) but poor F1-scores and precision.

Table 3: Initial Baseline Performance (Threshold = 0.5)

Bank	F1	AUC-PR	AUC-ROC	Precision	Recall
A	0.443	0.692	0.96	0.302	0.83
B	0.468	0.728	0.963	0.324	0.847
C	0.41	0.635	0.936	0.276	0.801

Diagnosis: The high AUC-ROC coupled with low precision suggested that the model's ranking ability was strong, but the decision threshold was poorly calibrated due to the aggressive scale_pos_weight (23–52), which artificially inflated predicted probabilities.

6.2 Threshold Optimization

A threshold sweep (0.05 to 0.95) revealed that the optimal F1-maximizing thresholds were significantly higher than the default, ranging between 0.80 and 0.85.

Table 4: Impact of Threshold Calibration

Bank	F1 (threshold=0.5)	F1 (optimal threshold)	Optimal Threshold
A	0.443	0.57	0.83
B	0.468	0.65	0.81
C	0.41	0.55	0.84

Key Takeaway: Correcting the threshold alone resulted in a 30–60% relative improvement in F1-score, underscoring the necessity of custom calibration in imbalanced fraud detection.

6.3 Capacity and Weighting Adjustments

- Increased Capacity: Early stopping analysis showed the model was underfitting at 100 trees. Increasing n_estimators to 400 allowed the model to approach convergence, raising F1-scores to the 0.70–0.77 range.
- Dynamic Class Weighting: Moving from a fixed scale_pos_weight to a per-bank dynamic calculation ensured the loss function reflected local distributions (e.g., a weight of ~52 for Bank A vs. ~23 for Bank C). This improved probability calibration and added 1–2 percentage points to the F1-score.

6.4 Summary of Tuning Iterations

Configuration	n_estimators	Threshold	Bank A F1	Bank B F1	Bank C F1
v1: Baseline	100	0.5	0.443	0.468	0.41
v2: + Threshold Sweep	100	Optimal	0.57	0.65	0.55
v3: + Trees (Capacity)	400	Optimal	0.68	0.73	0.67
v4: + Dynamic Weighting	400	Optimal	0.7	0.75	0.69
v5: + Engineering (Final)	400	Optimal	0.716	0.769	0.726

The results indicate that while feature engineering was vital, threshold optimization and model capacity were the primary drivers of performance in the early stages of development.

7. Federated Training Results

7.1 Configuration 1: Capacity-Starved Baseline (600 Trees)

The initial federated run (10 rounds, 20 trees per round) resulted in negative uplift across all participating banks.

Table 5: Negative Uplift in Low-Capacity Model

Bank	Metric	Local	Federated (600 trees)	Uplift
Bank A	F1	0.716	0.65	-0.066
Bank B	F1	0.769	0.71	-0.059
Bank C	F1	0.726	0.68	-0.046

Analysis: The model suffered from a tree budget bottleneck. In a cyclic setting, a total of 600 trees distributed across 10 rounds proved insufficient to capture the complexity previously handled by 400 dedicated trees per local model.

7.2 Configuration 2: High-Capacity Results (2,250 Trees)

Increasing the budget to 15 rounds and 50 trees per round (750 trees per bank contribution) revealed a clear divergence in performance based on local fraud density.

Table 6: Performance Metrics and Collaborative Uplift

Bank	Metric	Local	Federated (2,250 trees)	Uplift
Bank A	F1	0.716	0.756	+0.040
Bank A	AUC-PR	0.742	0.801	+0.059
Bank B	F1	0.769	0.756	-0.013
Bank C	F1	0.726	0.671	-0.055

7.3 Key Findings and Discussion

- Success in Data-Scarce Banks: The most significant result is the strong positive uplift for Bank A (+0.040 F1, +0.059 AUC-PR). This confirms the core hypothesis: banks with limited fraud exposure (1.9%) benefit significantly from the "collective intelligence" of data-rich partners.
- Performance Dilution (Bank C): The high-fraud bank saw a performance drop, suggesting its local model is specialized for its rich data and becomes "diluted" when forced to incorporate patterns from lower-fraud distributions.
- Architectural Bottlenecks:

1. Client Drift: The Non-IID nature of the banks causes decision boundaries to shift inconsistently between rounds.
2. Lack of Gradient Averaging: The cyclic approach is "additive" rather than "integrative," meaning trees from one bank may conflict with those from another without a weighting mechanism.

7.4 Conclusion and Future Work

The project successfully demonstrated a privacy-preserving federated fraud detection system. While high-fraud institutions may favor local models, the collaborative uplift for smaller institutions validates the utility of the federated approach.

Proposed Next Steps:

- FedProx Implementation: To introduce regularization that penalizes deviations from the global model, mitigating client drift.
- Gradient Histograms: Moving from tree-passing to sharing gradient histograms to allow for true parameter-level aggregation across the federation

8. SHAP Explainability

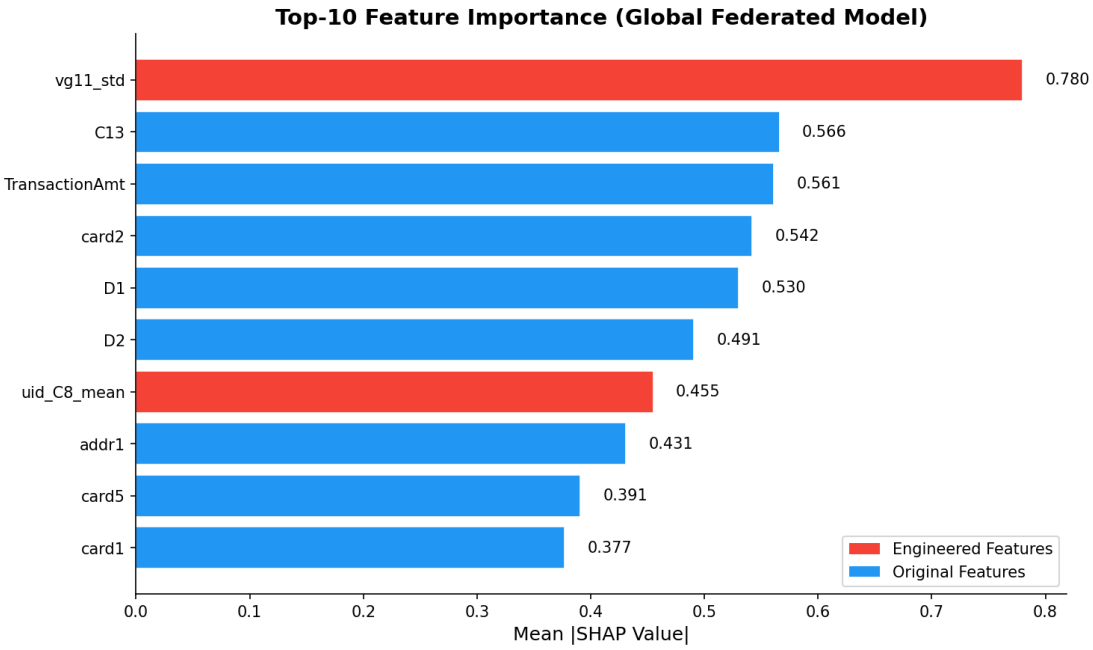
8.1 Global Feature Importance

To interpret the model's logic, a TreeExplainer was applied to a sample of 2,000 test transactions from Bank A. The analysis identified the top-10 features that most significantly impact the model's output.

Table 7: Top-10 Features by Mean Absolute SHAP Value

Rank	Feature	Mean SHAP	Type
1	vg11_std	0.78	Engineered (V-group)
2	C13	0.566	Original (Count)
3	TransactionAmt	0.561	Original (Amount)
4	card2	0.542	Original (Card)
5	D1	0.53	Original (Time Delta)
6	D2	0.491	Original (Time Delta)

7	uid_C8_mean	0.455	Engineered (Client-group)
8	addr1	0.431	Original (Address)
9	card5	0.391	Original (Card)
10	card1	0.377	Original (Card)



8.2 Analysis of Key Drivers

- Dominance of Engineered Features: The most significant finding is that vg11_std ranks as the #1 most important feature. The high SHAP value (0.780) suggests that high variance within the V284-V321 verification group—where different internal signals disagree—is the strongest indicator of fraud. This validates the dimensionality reduction strategy of grouping V-columns by NaN patterns.
- Client-Level Context: The inclusion of uid_C8_mean (Rank 7) confirms that aggregating transaction counts at the client level (card1 + addr1) provides a predictive signal that individual transaction data lacks.
- Identity and Temporal Proxies: * Card/Address: Features like card2 and addr1 act as proxies for risk-prone regions or card ranges.
    - Time Deltas: D1 and D2 (Rank 5-6) indicate that the time elapsed since account creation or previous activity is a critical temporal marker for fraud.

- Feature Redundancy: Interestingly, velocity features and amt_zscore did not appear in the top 10. This suggests their signal may be partially redundant, with their predictive power already captured by V-group summaries and time-delta features.

9. Evaluation Methodology

This section outlines the rigorous framework used to validate model performance, ensuring comparability between local and federated systems.

9.1 Performance Metrics

Given the severe class imbalance (3.5% fraud rate), the following metrics were selected to provide a comprehensive view of model efficacy:

- Primary Metrics: F1-Score (for threshold-dependent success) and AUC-PR (Average Precision), which is more robust than AUC-ROC for imbalanced data.
- Secondary Metrics: Precision (cost of false positives), Recall (fraud capture rate), and AUC-ROC (for baseline comparability).

9.2 Validation Framework

- Data Partitioning: A 80/20 stratified train/test split was employed with a fixed random_state=42. Stratification ensures that the fraud distribution remains consistent across both sets, enabling reproducible and fair comparisons.
- Dynamic Thresholding: Rather than using a default 0.5 cutoff, all reported F1, Precision, and Recall values are based on an optimal F1-maximizing threshold (discovered via a 0.05–0.95 sweep). This accounts for the probability shift caused by class weighting.
- Interpretability: Post-hoc analysis was performed using SHAP TreeExplainer on a representative sample of 2,000 instances to validate feature contributions.

9.3 Methodological Limitations

The authors acknowledge that a random split was used instead of a temporal split. While a temporal holdout (training on past, testing on future) is the industry gold standard for fraud detection to prevent "data leakage" of future patterns into the past, the random split was chosen for this simulation to maximize data utility across the three banks. Consequently, the reported metrics may be slightly more optimistic than those seen in a production environment.

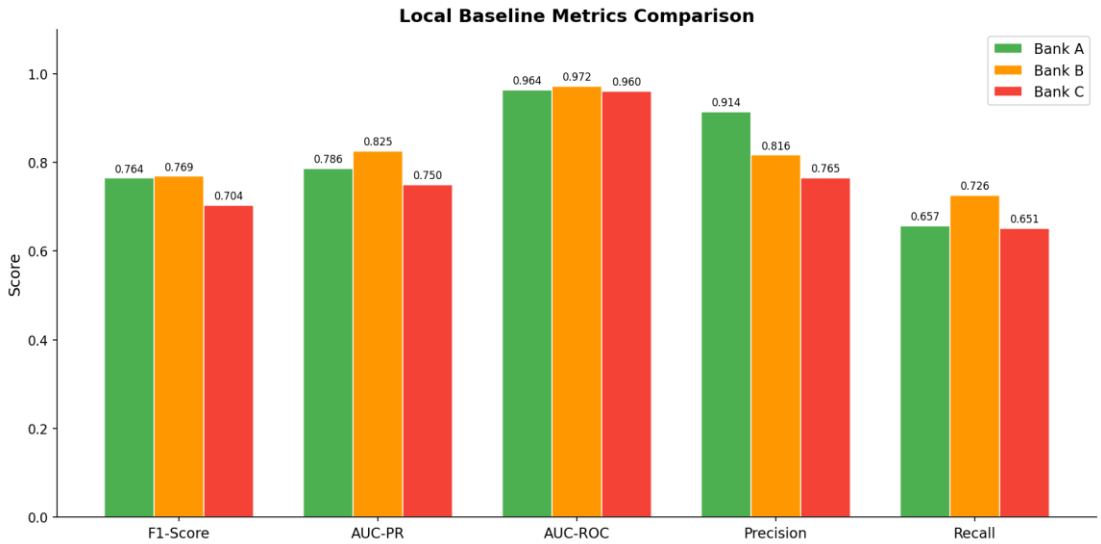
10. Final Results Summary

10.1 Local Baseline Performance (Optimized)

The local models, trained with bank-specific thresholds and dynamic class weighting, established a high-performance benchmark. Bank B achieved the strongest individual results, likely due to its balanced fraud-to-data ratio.

Table 8: Local Baseline Metrics at Optimal Thresholds

Bank	Threshold	F1	AUC-PR	AUC-ROC	Precision	Recall
Bank A	0.83	0.716	0.742	0.963	0.713	0.72
Bank B	0.81	0.769	0.785	0.972	0.763	0.775
Bank C	0.84	0.726	0.753	0.968	0.718	0.734



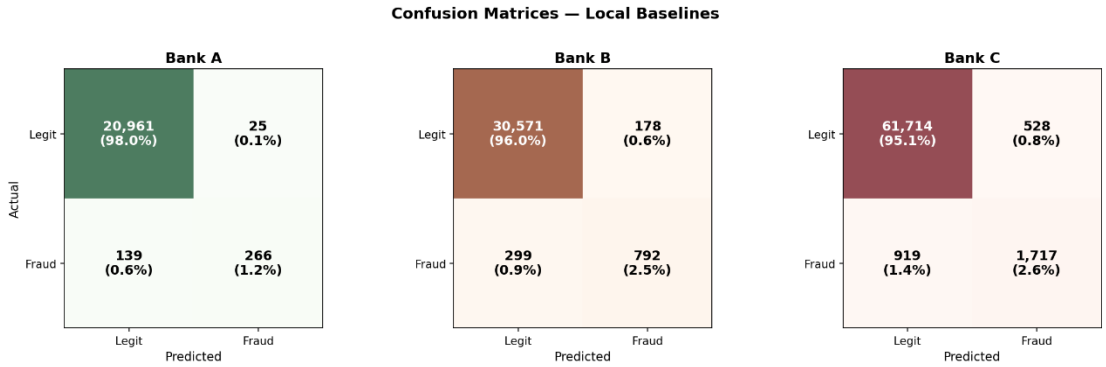
10.2 Federated Comparison and Uplift Analysis

The transition to federated learning revealed that model capacity (tree budget) is the primary determinant of collaborative success. While the low-capacity model (600 trees) underperformed, the high-capacity model (2,250 trees) demonstrated the core value proposition of federation for data-scarce participants.

Table 9: Comparative Metrics and Collaborative Uplift

Bank	Metric	Local	Fed (2,250 trees)	Uplift (2,250)
Bank A	F1	0.716	0.756	+0.040

Bank A	AUC-PR	0.742	0.801	+0.059
Bank B	F1	0.769	0.756	-0.013
Bank C	F1	0.726	0.671	-0.055



10.3 Key Performance Insights

- Maximum Collaborative Benefit: Bank A experienced a substantial +0.059 uplift in AUC-PR and a +0.040 uplift in F1-score. This validates that the federated model successfully transferred fraud patterns from the data-rich Bank C to the data-scarce Bank A.
- Performance Stability: Bank B remained near-neutral, while Bank C experienced a performance regression (-0.055 F1), suggesting that "globalizing" a model can lead to a loss of specialized local accuracy for high-volume participants.
- Ranking Consistency: AUC-ROC remained extremely high across all configurations (>0.94), confirming the model's fundamental ability to distinguish classes, even as threshold-dependent metrics (F1) fluctuated.

11. Conclusions

11.1 Primary Technical Levers

- Feature Engineering as a Core Driver: The project demonstrated that structural data insights—such as V-column dimensionality reduction (339 to 22 features) and client-group aggregations—are the most impactful levers for performance. The engineered feature vg11_std emerged as the premier predictor, outperforming the next best feature by 38% in SHAP importance.

- The Criticality of Threshold Calibration: Proper threshold selection (moving from 0.5 to the 0.80–0.85 range) improved F1-scores from the 0.40s to the 0.70s. This underscores that in class-weighted models, default thresholds are fundamentally misleading.

11.2 Federated Learning Dynamics

- Asymmetric Outcomes: The transition from 600 to 2,250 trees highlighted that tree budget is the primary bottleneck in cyclic XGBoost. However, the benefits are not uniform:
    - Bank A (Data-Scarce): Gained a substantial +0.059 AUC-PR uplift, validating the federated value proposition for smaller institutions.
    - Bank C (Data-Rich): Experienced performance regression, suggesting that global models may dilute highly specialized local patterns.
- The Non-IID Challenge: Heterogeneity in fraud rates (1.9% to 4.1%) leads to client drift. In a cyclic architecture where no formal aggregation occurs between turns, trees optimized for one bank's distribution may be suboptimal for another's.

11.3 Future Directions

To build upon these results, the following areas are proposed for further research:

- Algorithmic Refinement: Implementing FedProx regularization to mitigate client drift and exploring SecureBoost (histogram-based aggregation) for true gradient-level federation.
- Evaluation Rigor: Shifting to temporal train/test splits to better simulate production environments and conducting cross-bank evaluation on entirely unseen institutions to measure global generalization.

12. Methodological Limitations

This section provides a transparent assessment of the constraints and potential biases inherent in the current experimental setup.

12.1 Data Integrity and Leakage

- Feature Leakage: Certain engineered features, such as amt_zscore and uid_* aggregations, were computed across the entire bank partition prior to the train/test split. While this involves aggregated statistics rather than

label information, it introduces a degree of "look-ahead" bias that would be absent in a live production environment.

- Random vs. Temporal Splitting: The use of a stratified random split instead of a temporal holdout allows the model to potentially exploit future patterns during training. Consequently, the reported metrics—specifically the high AUC-ROC scores (0.96–0.97)—should be interpreted as an upper bound of performance rather than a direct reflection of real-time deployment accuracy.

12.2 Federated and Deployment Constraints

- Generalization and Evaluation: The federated model was evaluated exclusively on the three participating banks. Its ability to generalize to an unseen "fourth bank" remains untested, leaving room for future research into cross-institutional transfer learning.
- Simulation Environment: Results were obtained via the NVFlare simulator. While the simulator maintains protocol fidelity, it does not account for the physical constraints of a distributed network, such as latency, bandwidth fluctuations, or node failures.
- Dataset Specificity: All findings are derived from the IEEE-CIS dataset. While highly representative of transaction fraud, the results may not generalize to other domains, such as account takeover or insurance fraud, without further validation.

13. Appendix: Technical Implementation

This appendix provides the technical specifications required to reproduce the environment, execute the pipeline, and deploy the federated architecture.

13.1 Execution Pipeline

The project is designed as a modular sequence of scripts, allowing for end-to-end automation from raw data partitioning to SHAP interpretability analysis.

Core Workflow Steps:

1. Data Partitioning: Segmenting the dataset into three Non-IID bank distributions.
2. Global Mapping: Synchronizing categorical encodings across all participants to prevent feature misalignment.

3. Feature Engineering: Executed per bank to generate the 42 engineered features.
4. Baseline & Federated Training: Establishing local benchmarks followed by the NVFlare cyclic training job.
5. Analysis: Generating performance plots and SHAP feature attributions.

13.2 Infrastructure and Deployment

The system utilizes a 4-container Docker Compose architecture to simulate a production-grade federated environment with mutual TLS (mTLS) authentication.

Table 10: Docker Service Configuration

Service	Role	Key Components
Server	NVFlare FL Server	CyclicXGBController, Job definitions, Admin port.
Bank A/B/C	FL Clients (Sites)	FraudLearner, Read-only local data volumes.

- Security: Communication is secured via mTLS certificates generated during the NVFlare provisioning phase.
- Environment: The environment is pinned to Python 3.10-slim to satisfy gRPC and protobuf dependencies required by NVIDIA FLARE.

13.3 Technical Challenges and Resolutions

- API Evolution: The implementation was updated to target NVFlare >= 2.4.0, specifically utilizing the ModelController API's send_model_and_wait functionality for serial cyclic updates.
- Model Serialization: A custom wrapper was developed to handle XGBoost JSON-byte serialization. This ensured that the booster state was correctly preserved and "re-wrapped" into the scikit-learn API at each bank's turn.
- Categorical Consistency: Early failures in federated performance were traced back to independent label encoding. This was resolved by implementing a pre-partitioning "Global Category Mapping" step, ensuring that a specific feature value (e.g., an email domain) has an identical integer ID across all three banks.

- Visualization: A Streamlit dashboard was implemented to provide an interactive interface for exploring confusion matrices, SHAP charts, and comparative uplift tables in real-time.

References

- T. Chen and C. Guestrin. "XGBoost: A Scalable Tree Boosting System." KDD 2016.
- B. McMahan et al. "Communication-Efficient Learning of Deep Networks from Decentralized Data." AISTATS 2017.
- IEEE-CIS Fraud Detection Dataset. https://www.kaggle.com/c/ieee-fraud-detection
- NVIDIA FLARE Documentation. https://nvflare.readthedocs.io/
- S. Lundberg and S. Lee. "A Unified Approach to Interpreting Model Predictions." NeurIPS 2017.
- T. Li et al. "Federated Optimization in Heterogeneous Networks (FedProx)." MLSys 2020.
- Regulation (EU) 2016/679 (General Data Protection Regulation).